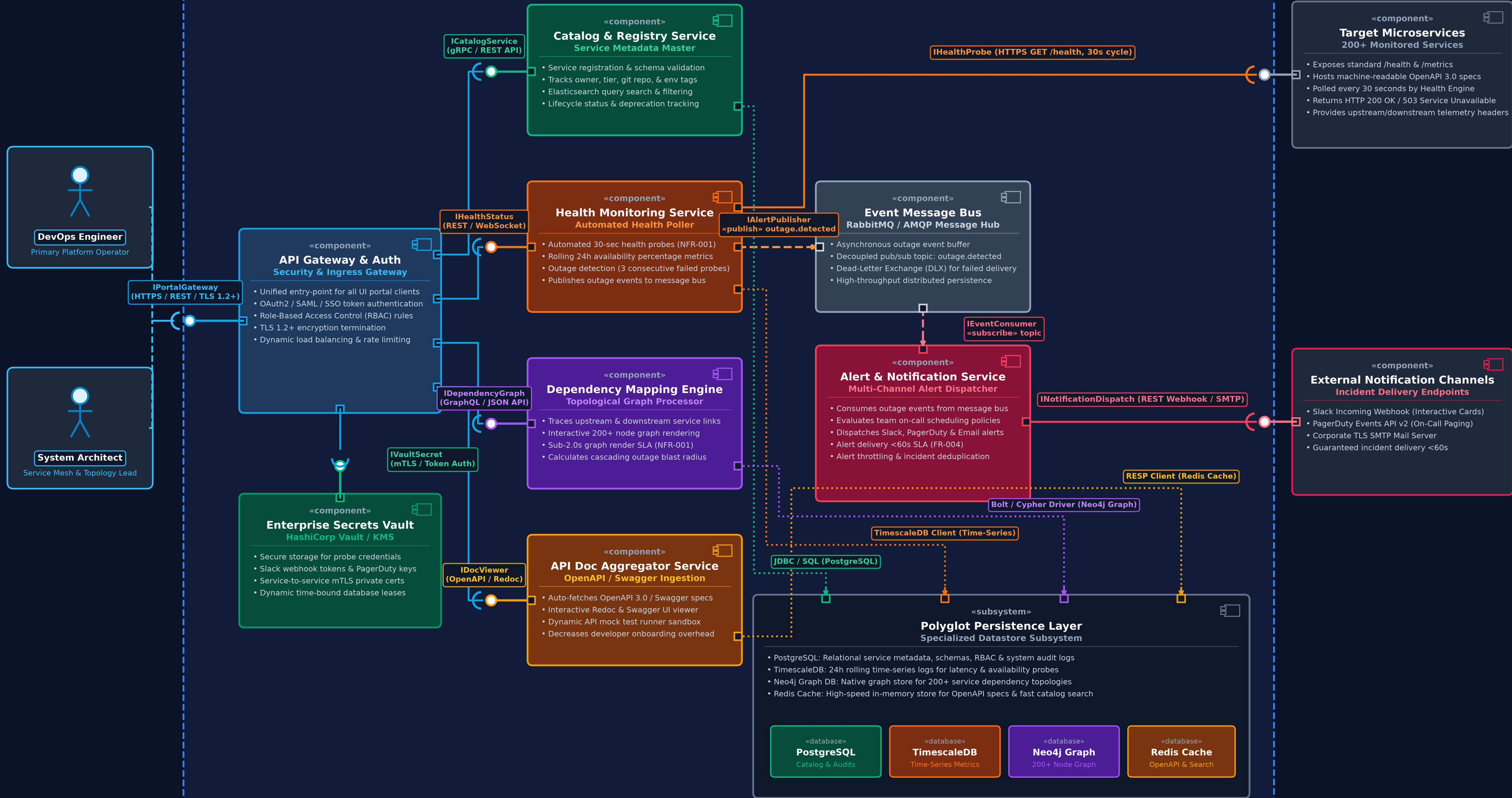


INTERNAL MICROSERVICE CATALOG & HEALTH PORTAL

UML 2.5 Component Diagram — Event-Driven Microservices Architecture (Problem Statement #42)

Student: Subramani B M | SRN: PES1UG24CS473 | Section: H | Dept. of CSE, PES University

«subsystem» Internal Microservice Catalog & Health Portal Core Runtime Platform



UML 2.5 NOTATION LEGEND (EVENT-DRIVEN ARCHITECTURE FOR PROBLEM STATEMENT #42):

- Component Box: Autonomous deployable service with defined business scope & encapsulated domain logic.
- Socket (Half-Circle): Required Interface denoting a consumer dependency on an upstream service contract.
- Dashed Arrow (---): Asynchronous publish / subscribe messaging decoupling event producers & consumers.
- Ball (Lollipop): Provided Interface exposing public synchronous gRPC / REST API contracts.
- Assembly Connector: Direct ball-in-socket binding coupling provided and required interface ports.
- Dotted Arrow (..): Polyglot database persistence driver binding (PostgreSQL, TimescaleDB, Neo4j, Redis).

Architectural Pattern Selection & Component Modeling Justification

Student: Subramani B M | SRN: PES1UG24CS473 | Section: H | Problem Statement #42: Internal Microservice Catalog & Health Portal

Architecture Selection: “We chose Microservices Architecture (Event-Driven with API Gateway) for the Internal Microservice Catalog & Health Portal System.”

1. Architectural Choice

We evaluated three candidate architectural styles: *Layered Architecture* was rejected because high-frequency polling I/O locks presentation/business threads and creates a monolithic deployment bottleneck. *Client-Server Architecture* was rejected due to a centralized single-point-of-failure under concurrent multi-service telemetry. **Microservices Architecture** was selected for its fine-grained horizontal scalability, independent service lifecycles, and event-driven decoupling between monitoring pingers and alerting channels.

2. Two Specific Scenario-Related Reasons for Selection

a. Decoupled Scaling & Workload Isolation for High-Frequency Health Probing (FR-001 vs. NFR-001): The system executes continuous health check probes every 30 seconds across hundreds of microservices and calculates 24-hour rolling availability. Network timeouts from degraded downstream services generate heavy I/O spikes. Decoupling the *Health Monitoring Service* from the *Dependency Mapping Engine* ensures background probe latency never degrades the interactive rendering SLA of the 200-node dependency graph (< 2s under NFR-001).

b. Autonomous Evolution & Fault Isolation in Alerting & Documentation (FR-004 & FR-005): The portal integrates with external notification endpoints (Slack Webhooks, PagerDuty, SMTP) and ingests diverse OpenAPI/Swagger 3.0 schemas across developer teams. Isolating the *Alerting & Notification Service* and *API Doc Aggregator* ensures external rate limits or invalid Swagger specs cannot crash the core service catalog or halt background health probes.

3. Security Advantage

Centralized Ingress Gateway & Encrypted Secrets Isolation (NFR-002): All client interactions are terminated at an edge *API Gateway & Authentication Service* enforcing enterprise SSO (OAuth 2.0 / SAML), TLS 1.2+ mutual encryption, and RBAC. Health probe credentials and webhook secrets are stored in an encrypted *Enterprise Secrets Vault (HashiCorp Vault/KMS)* accessible only via mTLS by backend workers, preventing plaintext exposure and protecting internal topology data.

4. Performance Benefit

Asynchronous Messaging & Specialized Polyglot Persistence (FR-003, FR-004, NFR-001): An *AMQP/RabbitMQ Event Broker* decouples outage detection from alerting, guaranteeing notification dispatch within 60s without stalling probe workers. The system leverages *Polyglot Persistence*: a dedicated Graph Database (**Neo4j**) performs sub-second traversals across 200+ nodes to achieve the < 2s graph render benchmark, while **TimescaleDB** handles time-series metrics and **Redis** accelerates catalog search.

5. Component & Interface Specification Summary

Component Name	Provided Interface (Ball)	Required Interface (Socket)	Core Architectural Responsibility
API Gateway & Auth	IPortalGateway (HTTPS/REST)	ICatalogService, IHealthStatus, IDependencyGraph, IDocViewer	TLS termination, SSO OAuth2/SAML validation, RBAC, traffic routing
Catalog & Registry	ICatalogService (gRPC/REST)	JDBC / SQL (PostgreSQL)	Searchable service registry, metadata management, elastic indexing
Health Monitoring	IHealthStatus (WebSocket/REST)	IHealthProbe (HTTPS GET), IAlertPublisher (AMQP)	30s health pings, rolling 24h availability, outage detection (3 fails)
Dependency Engine	IDependencyGraph (GraphQL/JSON)	Bolt / Cypher (Neo4j), ICatalogService	Auto dependency mapping, 200+ node interactive graph (<2s render)
Alerting & Notification	IAlertConfig (REST)	IEventConsumer (AMQP), INotificationChannel (Slack/SMTP)	Outage event evaluation and multi-channel dispatch within 60s (FR-004)
API Doc Aggregator	IDocViewer (OpenAPI / Redoc)	ISpecFetcher (Git/HTTP), Redis Driver	OpenAPI 3.0 schema parsing, endpoint documentation, mock sandbox